DECK 05 · WEEK 5

Escrow, DeFi & NFTs

Exercise 8 is your hardest program. Exercises 9 and 10 expand your integration skills for capstone-ready delivery.

ONE HEAVY PROGRAM + TWO LIGHTER INTEGRATIONS

This week rounds out your toolkit before capstone.

Exercise 8 · Escrow

Build trustless token exchange with `make`, `take`, and `cancel`.

HARDEST PROGRAM

Exercise 9 · DeFi

CLI reads Pyth prices and Jupiter quote, then computes spread.

QUOTE + ANALYSIS

Exercise 10 · NFTs

Mint a collection and members with Umi + Metaplex Token Metadata.

COLLECTION TOOLING

SUPPORT SHIFT

No AI guidance boxes from here on. The prompting and verification workflow is now fully yours.

ESCROW PROGRAM

Trustless swap.

MAKE · TAKE · CANCEL

TWO PARTIES, ONE PROGRAMMED GUARANTEE

Alice deposits Token A. Bob fulfills with Token B. Program enforces atomic exchange.

INSTRUCTIONS

01 `make` : lock maker Token A in PDA-controlled vault.

02 `take` : transfer Token B to maker and release vault Token A to taker atomically.

03 `cancel` : maker reclaims Token A and closes escrow.

STATE TO STORE

01 Maker pubkey for authority and Token B destination.

02 Token A mint + Token B mint addresses.

03 Requested amount for Token B side.

04 Vault reference and escrow PDA metadata.

LONG ACCOUNT LISTS ARE NORMAL HERE

Escrow combines two token transfers and account closures, so constraints must be precise.

`make` accounts

maker signer/payer, escrow PDA init, vault token account init, maker token A source, mint A, token program.

`take` accounts

taker signer, escrow, vault, maker token B destination, taker token A destination, taker token B source, both mints, token program.

`cancel` accounts

maker signer, escrow, vault, maker token A destination, token program; then close vault + escrow.

PDA authority

Vault token authority must be PDA-controlled so withdrawals only happen via program logic.

THIS WEEK'S MOST IMPORTANT TEST SUITE

Start from failure paths and balance reconciliation, not only happy path execution.

CASE	EXPECTED BEHAVIOR	SIGNAL
Make → Take	Alice receives Token B, Bob receives Token A, vault closes.	Balances update and accounts closed.
Make → Cancel	Alice gets Token A back, escrow closes.	Original balance restored.
Double-take	Second take fails because escrow is closed.	Clear error, no token movement.
Take after cancel	Take fails after cancellation.	Closed-account failure path.
Wrong mint / wrong authority	Invalid taker token and non-maker cancel both fail.	Constraint enforcement confirmed.
Balance reconciliation	Total token supply conserved across all tests.	No hidden mint/burn side effects.

GUARANTEE TRUSTLESS BEHAVIOR

Most critical failures are authority, mint validation, and close-account handling.

VAULT AUTHORITY

If maker is vault authority instead of PDA, escrow can be bypassed entirely.

MINT VALIDATION

Check taker sends the expected mint, or worthless tokens can drain escrowed assets.

OWNER VS AUTHORITY

Token account owner is Token Program; token authority is your PDA. Do not confuse them.

CLOSURE RULES

On completion/cancel, close vault and escrow and send rent lamports to intended recipient.

DEFI BASICS

Read prices. Compare routes.

PYTH · JUPITER · SPREAD

QUOTE ONLY. NO SWAP EXECUTION.

Build one TypeScript tool that fetches, checks freshness, and compares two price sources.

01 Fetch SOL/USD from Pyth (devnet feed).

02 Print confidence interval and staleness warning if > 30s old.

03 Get SOL→USDC quote from Jupiter on mainnet.

04 Compute implied Jupiter price and spread vs Pyth.

EXAMPLE CLI SHAPE

```
SOL/USD (Pyth):      $148.23 +/- $0.12
Last updated:        2s ago
SOL/USDC (Jupiter): $148.15 (for 1 SOL)
Spread:              0.054%
```

NETWORKS, DECIMALS, API VERSIONS

Most broken outputs come from environment mismatch and unit conversion mistakes.

PITFALL	CAUSE	FIX
Pyth account not found / stale	Wrong feed id or network mixup.	Lookup current feed ID for the target network.
Jupiter implied price off	Raw units not converted by decimals.	Use SOL 9 decimals, USDC 6 decimals in math.
Request failures	Outdated Jupiter endpoint schema.	Confirm current API format in docs.
Nonsense comparison	Using devnet liquidity for quote.	Keep Jupiter quote on mainnet only.

NFTS & METADATA

Mint and verify assets.

TOKEN + METADATA = NFT EXPERIENCE

On Solana, NFT fungibility is defined by mint supply and metadata structure.

NFT token primitive

Mint supply 1, decimals 0. Non-fungible behavior comes from metadata, not special token math.

Chosen standard

Use Metaplex Token Metadata with Umi for widest ecosystem support.

Alternatives

Token-2022 metadata extension and Metaplex Core exist, but tooling coverage differs.

Collection goal

One parent collection NFT + three member NFTs with verified collection references.

SIGNER MODEL MATTERS

Umi signer/connection model differs from wallet-adapter and web3.js scripts.

01 Create Umi instance on devnet.

02 Load/generate keypair signer for scripts.

03 Create collection NFT, then member NFTs with verification signature.

04 Fetch collection items and print name/image/attributes.

URI RULE

Metadata URI must point to valid public JSON. Broken JSON/URL means missing image/attributes in wallets.

COLLECTION VERIFICATION

If member NFTs are unverified, collection authority signature was not included.

COMPLETION RUBRIC

Use this checklist as your capstone readiness gate.

ESCROW + TESTS

01 `make`, `take`, and `cancel` all function correctly.

02 Wrong mint / wrong authority / closed-account paths fail clearly.

03 Balances reconcile with no token creation/destruction.

INTEGRATIONS

01 DeFi CLI prints Pyth price + confidence + spread vs Jupiter quote.

02 NFT collection and three members minted and verified on devnet.

03 Script lists collection NFTs with image and attributes.

OPTIONAL DEPTH BEFORE CAPSTONE

Each stretch goal trains a production-level concern: liquidity, UX, or scale economics.

⚙ **ESCROW**

Add partial fills with proportional amounts, overflow-safe arithmetic, and rounding-aware state updates.

i **DEFI**

Integrate live Pyth + quote panel into your Week 4 React frontend (quote-only).

ϟ **NFTS**

Compare cost of 100 regular NFTs vs 100 compressed NFTs (Bubblegum + Merkle tree).

FINAL WEEK BEFORE CAPSTONE

Escrow proves program depth. DeFi and NFTs prove ecosystem fluency. You're ready to combine both into one product.

Protocol engineering

You can design and test multi-party token flows with strict account constraints.

Market integrations

You can consume real-time pricing and route quotes safely for product decisions.

Digital asset tooling

You can mint, verify, and query NFT collections using modern Metaplex tooling.

Capstone readiness

You can choose escrow, DeFi, NFTs, or combine them in one complete dApp.